

Multi-Vendor E-Commerce Marketplace

- Complete System Architecture

Project: Jumia-like Multi-Vendor Marketplace

Stack: Node.js (NestJS + TypeScript), MongoDB + PostgreSQL, Redis

Architecture: Modular Monolith (Domain-Driven Design)

Table of Contents

- 1. [System Overview](#)
- 2. [High-Level Architecture](#)
- 3. [Database Design](#)
- 4. [Third-Party Integrations](#)
- 5. [Deployment Strategy](#)

1. System Overview

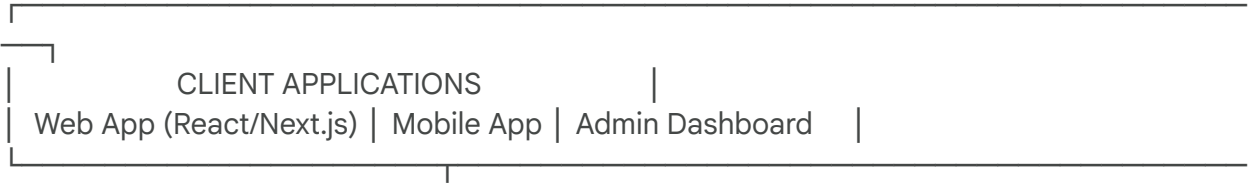
Business Rules Summary

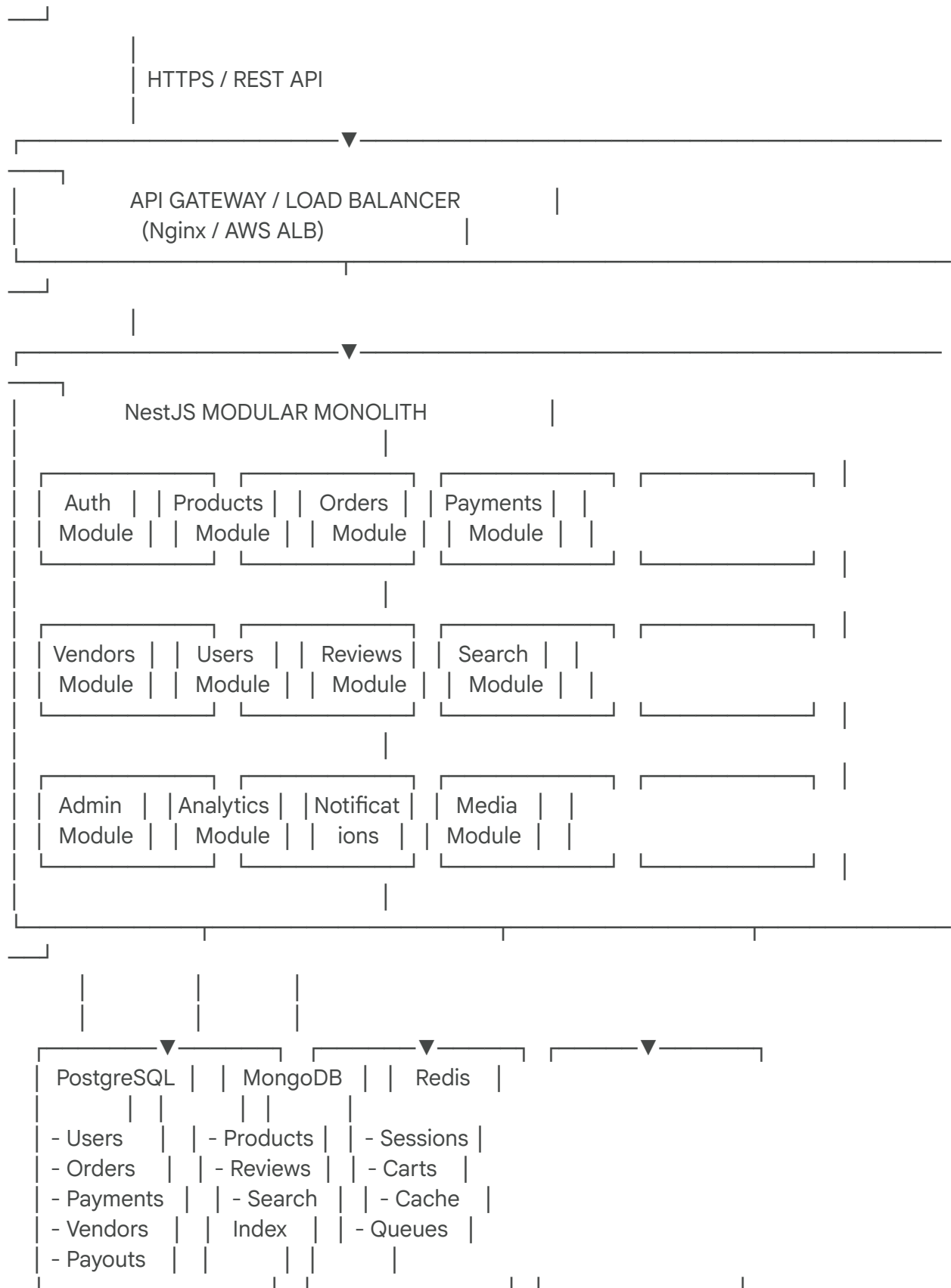
- **Commission Model:** Vendor subscription fee + smaller commission per sale.
- **Vendor Onboarding:** Manual admin approval with business document verification.
- **Payout Schedule:** Monthly batch payouts (1st of every month).
- **Product Model:** Multi-SKU with variants (Color, Size, Material, etc.).
- **Order Flow:** Single order with sub-orders per vendor.
- **Payment Methods:** Card (Paystack/Flutterwave), Bank Transfer, Cash on Delivery.
- **Refund Window:** 14 days post-delivery.
- **Delivery:** Hybrid model (vendor's own logistics or platform-provided).
- **Ratings:** Product, Vendor, and Delivery experience (all separate).

User Roles

- 1. **Customer** - Browse, purchase, review, manage orders.
- 2. **Vendor** - Manage products, inventory, orders, view analytics.
- 3. **Admin** - Platform oversight, vendor approval, dispute resolution, analytics.

2. High-Level Architecture





Why Modular Monolith?

- **Single deployable unit** - Easier DevOps and lower infrastructure costs.
- **Shared database transactions** - Ensures data integrity across different modules.
- **Clear module boundaries** - Can be easily extracted into microservices if the platform scales massively.
- **Faster development** - No complex inter-service communication overhead during the MVP phase.

3. Database Design

Database Split Strategy

1. **PostgreSQL (Transactional Data)**
 - Used for Users, Vendors, Orders, Payments, and Payouts.
 - Handles data requiring strict ACID compliance and complex relationships.
2. **MongoDB (Flexible Schema Data)**
 - Used for Products (with dynamic attributes per category) and Product Reviews.
 - Provides high performance for read-heavy catalogs and flexible document structures.
3. **Redis (In-Memory Cache)**
 - Manages User sessions, JWT blacklisting, and temporary Shopping Cart data.

4. Third-Party Integrations

- **Payments:** Paystack / Flutterwave.
- **Media Storage:** AWS S3 or Cloudinary for product images.
- **Notifications:** SendGrid (Email) & Termii (SMS).
- **Logistics:** Integration with DHL/GIG Logistics APIs for real-time tracking.
- **Search:** Elasticsearch for advanced product filtering.

5. Deployment Strategy

- **Containerization:** Dockerized services for consistent environments.
- **CI/CD:** Automated testing and deployment pipelines.
- **Infrastructure:** Cloud-hosted (AWS/DigitalOcean) with managed database instances.